



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 8: Bit Manipulations (Rotations, Multiplication, Extended & Bitwise Logical Operations)

1. Objectives

- Understand the concept and use of rotate and shift operations.
- Implement the basic multiplication algorithm.
- Perform extended arithmetic operations using processor flags.
- Apply bitwise logical operations in assembly programs.

2. Rotate Instructions

Rotate instructions shift the bits of an operand in a circular fashion, unlike shift instructions where bits are lost or replaced by zeros. The **CF** often plays a crucial role in these operations.

Instruction	Name	Effect on Bits	Carry Flag (CF)	Flag Impact
ROR	Rotate Right	Bits rotate right; no fill, MSB gets old LSB. CF = old LSB.	Old LSB	CF,OF
ROL	Rotate Left	Bits rotate left; no fill, LSB gets old MSB. CF = old MSB.	Old MSB	CF,OF
RCL	Rotate Left through Carry	Rotate including CF: bits shift left, CF gets old MSB, MSB gets old CF.	Old MSB	CF
RCR	Rotate Right through Carry	Rotate including CF: bits shift right, CF gets old LSB, LSB gets old CF.	Old LSB	CF

- **ROR (Rotate Right):** Rotates all bits to the right. The LSB moves into the *CF* and into the MSB position.
- **ROL (Rotate Left):** Rotates all bits to the left. The MSB moves into the *CF* and into the LSB position.
- **RCL (Rotate through Carry Left):** Rotates all bits and the *CF* to the left. The MSB moves into the *CF*, and the original *CF* moves into the LSB position.
- **RCR (Rotate through Carry Right):** Rotates all bits and the *CF* to the right. The LSB moves into the *CF*, and the original *CF* moves into the MSB position.



Example:

```
[org 0x0100]
jmp start
num: db 11001010b      ; 8-bit number
start:
    mov al, [num]
    rol al, 1
    ror al, 1
    rcl al, 1
    rcr al, 1
mov ax, 0x4c00
int 0x21
```

3. Multiplication Algorithm (Manual Approach)

A simple multiplication can be implemented using **repeated addition**.

Example:

```
[org 0x0100]

jmp start

multiplicand: db 13      ; 4-bit number, save space of 8-bits
multiplier:   db 5       ; 4-bit

result:       db 0       ; 8-bit result

start:
    mov cl, 4           ; how many times we need to run the loop
    mov bl, [multiplicand]
    mov dl, [multiplier]

    checkbit:
        shr dl, 1
        jnc skip
        add [result], bl    ; only add if CF IS SET

        skip:
            shl bl, 1        ; always shift the multiplicand
            dec cl
            jnz checkbit
mov ax, 0x4c00
int 0x21
```



4. Extended Operations

These operations are used when **results exceed register size**. This is achieved by linking multiple registers and utilizing the *Carry Flag (CF)* or *Overflow Flag (OF)*.

4.1. Extended Shift Operations

- **SHL/SAL:** Shift Left (multiply by 2)
- **SHR:** Shift Right (divide by 2)
- **SAR:** Shift Arithmetic Right (preserves sign bit)

```
[org 0x0100]
jmp start

num: db 8

start:
    mov al, [num]
    shl al, 1      ; Multiply by 2
    shr al, 1      ; Divide by 2

    mov ax, 0x4c00
    int 0x21
```

4.2. Extended Addition

To add two multi-word numbers, we add the least significant words first, then use the **ADC** (Add with Carry) instruction for the subsequent words.

```
[org 0x0100]
jmp start

    num1:  dw 0FFFFh
    num2:  dw 0001h
    high:  dw 0000h

start:
    mov ax, [num1]
    add ax, [num2]      ; AX = FFFF + 0001 → 0000, CF=1
    mov word [num1], ax
    adc word [high], 0   ; Add carry to next word

    mov ax, 0x4c00
    int 0x21
```



4.3. Extended Subtraction

To subtract two multi-word numbers, we use the **SBB** (Subtract with Borrow) instruction for the subsequent words.

```
[org 0x0100]
jmp start

num1:    dw 0000h
num2:    dw 0001h
high:    dw 0000h

start:
    mov ax, [num1]
    sub ax, [num2]
    mov word [num1], ax
    sbb word [high], 0        ; Subtract with borrow

mov ax, 0x4c00
int 0x21
```

4.4. Extended Multiplication

It is a series of additions/carry management to compute the final, larger product.

```
[org 0x0100]
jmp start

multiplicand: dw 0FFFFh
multiplier:    dw 0002h
result:        dd 0

start:
    mov ax, [multiplicand]
    mov bx, [multiplier]
    mul bx          ; DX:AX = AX * BX
    mov [result], ax
    mov [result+2], dx

mov ax, 0x4c00
int 0x21
```



5. Bitwise Logical Operations

Bitwise logical instructions perform operations on corresponding bits of the operands.

Instruction	Operation	Example
AND	Logical AND	AND AL, BL
OR	Logical OR	OR AL, BL
XOR	Exclusive OR	XOR AL, BL
NOT	Logical NOT (1's complement)	NOT AL

```
[org 0x0100]
jmp start
```

```
num1: db 0F0h
num2: db 0AAh
result: db 0
```

```
start:
    mov al, [num1]
    mov bl, [num2]

    and al, bl
    mov [result], al        ; Store AND result

    mov al, [num1]
    or al, bl
    mov [result], al        ; Store OR result

    mov al, [num1]
    xor al, bl
    mov [result], al        ; Store XOR result

    mov al, [num1]
    not al
    mov [result], al        ; Store NOT result

mov ax, 0x4c00
int 0x21
```

6. Lab Tasks

Task 1: Write a program to perform *rotate left* and *rotate right* on an 8-bit number.

Task 2: Implement multiplication using repeated addition (without using MUL).

Task 3: Perform extended addition and subtraction of two 16-bit numbers.

Task 4: Write a program that performs bitwise AND, OR, XOR, and NOT operations on two numbers and observe the result.